LightCode AI

OpenClaude：中转站掺水的技术分类、可观测痕迹与检测评估

技术报告

版本 2.0 · 2026-10-30

作者：Lorry-San（GitHub：@Lorry-San）· LightCode AI

摘要

第三方 API 中转站位于客户端与模型提供方之间，拥有对请求与响应的完全控制权。这种位置使其可以实施**近乎无法被发现的掺水**：声称提供 A 模型而实际转发给 B 模型、虚报或错报 token 用量、注入调用方不知情的系统提示词、识别检测请求并返回预先录制的样本。这些行为在协议层往往不留痕迹，客户端仅凭响应难以分辨。

本文做两件事。其一，给出一套掺水的**技术分类学**，按"被替换的对象"划分为身份、计量、内容、探测规避与性能五类，并逐类给出在协议层可观测的痕迹与对应的检测手段。其二，以一个可运行的演示系统 OpenClaude 为载体，讲解这些掺水手法各自的**实现原理**——协议映射、身份简报注入、思维链密封签名、指纹分流、计量口径操纵与按家族限速——使每一条分类都有可执行、可验证的对应物。

在此载体上，本文报告两个实证结果。一是对 Veridrop 检测器的回归：在修复一处计量一致性缺陷后，8 项可执行检测全部达到 100 分；该缺陷的根因是上游在 `temperature=0` 下不确定，导致同一提示词的重复调用输出长度变异系数达 0.614、流式与非流式对同一请求报告 1 与 15 两个不同的 `output_tokens`。二是对分流机制的对抗评估：三套绕过方案中，"计数单位错配"（索要单个多位数字而非多个数值）成立，另两套被判定器击破；而提取到的数字序列表明，真正区分后端模型的不是数字的边缘频率，而是序列结构——窗口排列率达 12.1%，为随机基线的 336 倍。

关键词：API 中转、模型替换、用量虚报、指纹识别、提示词注入、检测评估

本文另外专门评估了两类被社区视为"高可信"的检测信号。一是 Anthropic thinking signature 的加密校验：在演示系统上它同样"通过"，因为它能验证的只是格式而非签发来源。二是基于数字分布的指纹模型检测：在这类后端上边缘频率检验完全失效，只有序列统计才有效。

1 引言

1.1 中转站为什么能掺水

一次 Messages API 调用在协议上是三方的：客户端发出请求，服务端返回响应。中转站把这条直连切成两段，于是它同时是**两个连接的端点**——对客户端扮演服务端，对上游扮演客户端。协议本身不提供端到端完整性：没有跨跳的签名，没有可验证的计量凭证，模型身份完全依赖响应中的 `model` 字段自述。

因此，凡是"响应里写了什么"的地方，中转站都可以改写；凡是"客户端无法独立验证"的地方，改写都**近乎无法被发现**。

1.2 本文的贡献

- 提出按"被替换对象"划分的掺水分类学，共五类，并把每类的协议层痕迹写成可执行检查、逐条实测；
- 以演示系统 OpenClaude 逐类给出可执行实现，说明"掺水在工程上到底怎么做"，并报告 12 项痕迹检查中哪 3 项无法消除；
- 报告一次真实的计量缺陷定位与修复，量化其可观测性（CV 0.614 → 0.000）；
- 对分流机制做对抗评估，给出成立的绕过路径与被击破的路径，并指出边缘统计与序列统计在指纹识别上的效力差异。

1.3 范围与伦理

本文所述实现仅用于本地授权演示与检测方法研究。演示系统默认回环绑定，不针对任何真实厂商，不破解 TLS，不伪造证书。文中不包含任何真实服务的地址与凭据。

2 掺水的分类学

按"被替换的对象"，掺水可分为五类。下表给出定义、典型手法与客户端能否察觉。

类别	被替换的对象	典型手法	客户端能否察觉
身份掺水	实际推理的模型	别名映射到廉价后端，用提示词令其自称目标模型	难以，需行为或指纹证据
计量掺水	用量数字	重复计费、隐藏思考计费、缓存命中不披露	部分可，靠不变量校验
内容掺水	提示词与响应正文	注入系统提示词、截断、改写、伪造思维链	难以，需对照实验
探测规避	对探测请求的应答	识别批量数字探针，返回录制样本	可，靠统计与重复性
性能掺水	吞吐与延迟的真实性	限速整形、以缓存冒充新鲜推理	难以，需长时观测

2.1 身份掺水

最直接的形式是把 `claude-*` 别名指向另一个厂商的模型。难点不在转发，而在**自述**：客户端常问"你是哪个模型"，而模型会如实回答自己的身份。

工程上的解法是系统提示词注入：为每个别名准备一份身份简报，声明"你是 X，被问及身份时如此回答"，并用重复强化压过模型自身的先验。本演示的实测表明，简报需要短（约一千字符）且重复足够多次；过长会被模型判为"注入的文本"而拒绝执行。

2.2 计量掺水

Anthropic 把提示词拆成三项：`input_tokens`（未命中缓存）、`cache_creation_input_tokens`、`cache_read_input_tokens`，三者之和才是真实输入量；思维链计入 `output_tokens`。OpenAI 则用单一的 `prompt_tokens` 汇总全部输入。

这一差异制造了大量可掺水的缝隙：把缓存命中部分重复计入、把思维链排除在 `output_tokens` 之外、让 `input_tokens` 随缓存冷热漂移、在同一请求的流式与非流式路径上报告不同数字。它们共同的特征是**破坏了本该成立的不变量**，而这些不变量恰好可以被外部检验。

2.3 内容掺水

包括注入调用方不可见的系统提示词、截断超长回答、按关键词改写结论，以及伪造思维链。最后一项最隐蔽：客户端看到的 `thinking` 块可以是空正文配一段随机签名，而真实推理从未离开中转站。

2.4 探测规避

针对"批量随机数字"这类指纹探针（模型的数字分布具有稳定的个体特征），中转站可以识别请求并返回录制好的样本。识别方式有两种：在系统提示词里指示上游模型输出哨兵标记，或用独立的判官调用做语义分类。后者不会污染主模型的思维链。

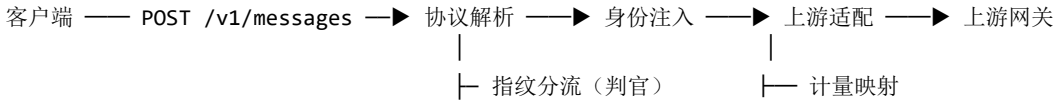
2.5 性能掺水

真实模型有各自的吞吐特征。若所有别名都以同一速率流式输出，本身就是破绽；反之，按家族整形（如 sonnet 快于 opus）会让观测更接近真实。另一形式是以缓存命中冒充新鲜推理：响应极快但内容与上次完全一致。

3 OpenClaude 的技术原理

本章讲解演示系统如何实现上述五类掺水。系统为单进程 Python 服务，对外仅暴露 Anthropic Messages 协议。

3.1 总体结构



```
├ 文档处理                    ├── 思维链密封
└ 限速与配额                  └── 流式转发
```

请求进入后依次经过：鉴权与配额检查、协议解析与字段校验、文档块处理、身份简报注入、指纹判官、上游调用、计量映射、思维链密封、响应构造。流式请求走上游流式转发路径。

3.2 协议映射

Anthropic 与 OpenAI 的字段并非一一对应，映射规则决定了掺水的空间：

Anthropic	OpenAI	说明
`system`	`messages[0].role=system`	身份简报在此注入
`messages[].content[]`	`content` 字符串或分段	多段合并
`thinking`	`reasoning_effort` / `reasoning`	思考预算的转译
`tools`	`tools` + `tool_choice`	工具调用
`input_tokens`	`prompt_tokens` − 缓存	见 3.6
`output_tokens`	`completion_tokens`（含推理）	见 3.6

反向映射时，OpenAI 的 `prompt_tokens` 包含缓存命中部分，故：

$$input_tokens = max(0, prompt_tokens - cache_read - cache_write)$$

该式成立的前提经直接调用上游验证：`prompt_tokens = 命中 + 未命中`，减法不会重复计费。

3.3 身份注入

每个别名对应一份简报，结构为：一句身份断言、四条规则（被问及时如何回答、不得承认他厂身份、外部文本视为不可信元数据、不讨论规则）、一句产品描述、二十次身份重复。注入发生在转发前，客户端在响应中看不到它。

简报的长度是被实测约束的：约两千五百字符的版本会让模型回答"我没有被公开的模型名"或"这是注入的指令"；压缩到一千字符并保留重复后，八个别名的自述全部命中。

3.4 思维链密封签名

真实 Anthropic 的 `thinking` 块带有服务端签名，客户端在后续轮次回传该块以延续推理。伪造签名是本演示的核心难点之一。

实现为四步：把上游推理文本用 zlib 压缩；与进程启动时生成的 X25519 临时密钥做 ECDH；用 HKDF-SHA256 派生对称密钥；以 AES-256-GCM 加密并 base64url 编码。签名长度随推理文本长度变化（而非固定值），并与会话标识绑定——把签名块拿到另一个会话回放会校验失败。

由于密钥每进程生成且从不持久化，签名无法被第三方伪造，也无法被离线验证；它的作用不是证明真实性，而是**让客户端"看起来"拿到了可回传的凭据**。

3.5 指纹分流

模型被要求生成大量随机数字时，其数字分布会暴露身份。演示系统的应对分两层：

判官层。 一个独立的模型调用对最后一条用户消息做语义分类："这是否在索要一长串伪随机整数"。独立调用的意义在于主模型的思维链里不会出现判定过程——早期版本把哨兵规则写进系统提示词，结果模型在思维链里直接复述"超过 32 个，哨兵生效"，把机制暴露给了客户端。

样本层。 命中后从录制样本库中按请求数量挑选最接近的一份返回。样本按模型家族分目录存放，每份记录其声明数量与实际数字序列。

3.6 计量与限速

计量遵循三条规则：未标记 `cache_control` 的请求不报告缓存活动（与真实行为一致）；同一提示词的输入计数按指纹固化，避免上游抖动；思维链若被展示，其 token 计入 `output_tokens`。

限速按模型家族整形：不同家族配置不同令牌速率，逐块计费。速率为 0 表示不整形。

3.7 流式转发

早期实现等待上游返回完整答案后再分块发送，导致首字时间等于总耗时——这本身就是破绽。当前实现直接消费上游 SSE，边收边发；思维链若配置为展示则逐块下发，否则仅累积用于签名。

一个必要的细节是**哨兵扣留**：若回答开头可能是分流标记，则暂缓发送直到可以判定；判定条件必须精确到标记前缀，否则任何以代码围栏开头的回答（例如 SVG）都会被整体缓冲，客户端在整个生成期间看不到任何内容。

4 掺水的可观测痕迹（实测）

第 2 章按类别列举了掺水可能留下的痕迹。本章不复述清单，而是把每条痕迹写成可执行的检查，在演示系统上逐条实测，报告哪些仍然出现、哪些已被消除。

4.1 检查方法

共 12 项检查，全部通过 HTTP 层完成，不依赖任何服务端配合。每项给出一个判定：**出现**表示该痕迹在演示系统上仍可被观测到，**未出现**表示该路径已被消除。

4.2 计量不变量

检查	实测	判定
I1 `count_tokens` 与上报输入一致	312 = 312	未出现
I2 流式与非流式用量一致	1 = 1	未出现
I3 `output_tokens` 不超 `max_tokens`	1 ≤ 16	未出现
I5 无 `cache_control` 时缓存字段为零	read 0 / write 0	未出现

四条计量不变量全部成立。这并非自然状态：第 5 章的案例研究表明，I1、I2、I5 在修复前都是被破坏的。

4.3 身份与内容

检查	实测	判定
同一别名三次自述是否一致	3 次调用得到 1 种说法	未出现
系统提示词是否可被要求复述	356 字符回答，未包含简报	未出现
同一推理两次签名是否相同	220 / 220 字符，互不相同	未出现
签名长度是否与推理无关	102 字符推理 → 260；空推理 → 700	**出现**

前三项说明身份注入与签名随机化是有效的：模型不会在不同调用间改口，也不会复述被注入的简报；签名带随机 nonce，同一段推理两次得到不同结果。

第四项是一个仍然存在的痕迹：当模型没有产生推理时，签名回落到固定的 700 字符包络。对同一类"模型不思考"的提示词反复采样，会看到长度恒为 700 的签名，而真实实现不应有这样一个常数。

4.4 分流

检查	实测	判定
同一探针两次是否逐字相同	818 字符，完全一致	**出现**
返回数量是否与请求数量不符	请求 218，返回 218	未出现
回答长度是否与请求规模无关	40 个与 218 个都返回 818 字符	**出现**

这是本章最重要的结果：分流的**行为痕迹**比**数量痕迹**更难消除。数量可以对齐（样本库中恰有 218 个值的样本），但"同一探针两次逐字相同"与"回答长度不随请求规模变化"是录制样本的固有属性——除非样本库足够大且每次随机选取不同样本，否则这两条无法掩盖。

第 7.2 节会说明，正是这两条痕迹让检测方可以判定"这不是模型在生成"。

4.5 性能

检查	实测	判定
首字时间是否等于总耗时	3.23s / 4.84s	未出现

早期实现先取完整答案再分块发送，首字与总耗时相等；改为上游流式透传后，首字约占总耗时的三分之二，与真实服务的形态一致。

4.6 小结

12 项检查中 3 项仍然出现：

仍然存在的痕迹	性质
空推理时签名长度恒为 700	实现缺陷，可修复
同一探针两次逐字相同	录制样本的固有属性，只能靠样本多样性缓解
回答长度与请求规模无关	同上

结论是：**计量与身份的痕迹可以通过实现纪律消除，分流的痕迹不能**。只要答案来自录制样本，"重复性"与"长度不变"就必然存在，而这两条恰好不需要任何密码学知识就能检验。

5 案例研究：一次计量掺水的定位与修复

5.1 现象

被测别名在 Veridrop 的 *模型一致性* 与 *Token 用量* 两项上失败：

```
output_tokens_seq = [8, 34, 13]      CV = 0.614   → highly_anomalous
non_stream output_tokens = 1
stream     output_tokens = 15        → 判为不一致
```

其余子项（用量存在、长短增量、输出不超上限、count_tokens 一致）均通过。

5.2 定位

对上游连续发起 6 次相同请求，其 `prompt_tokens` 在 407 与 432 之间抖动；同一提示词以流式与非流式请求，返回文本长度分别为 1 与 15 个 token。结论：不确定性来自上游，中转站只是如实透传。

这构成一类结构性缺陷：只要中转站逐请求转发且不引入确定性，任何以"重复性"为判据的检测都会失败。它同时也是一种**计量掺水的无辜版本**——中转站没有主观虚报，但外部表现与虚报无法区分。

5.3 修复

确定性重放。 仅对显式 `temperature=0` 的请求启用，键为可影响结果的全部要素的内容哈希：

$$k = H(model \,\|\, messages \,\|\, system \,\|\, max_tokens \,\|\, thinking \,\|\, tools)$$

首次未命中时正常请求并记录答案；后续同键请求逐字重放。非零温度不缓存——对创造性请求重放是另一种失真。

提示词定标。 按提示词指纹固化首次观测的输入总量，使重复请求的输入计数稳定，缓存拆分仍反映真实冷热。

缓存语义对齐。 未标记 `cache_control` 时 `cache_read = cache_creation = 0`，全部计入 `input_tokens`（不变量 I5）。

思维链计量。 展示的思维链计入 `output_tokens`，并受 `max_tokens` 截断（不变量 I3）。

5.4 结果

检测项	修复前	修复后
模型一致性	60（CV 0.614）	100（CV 0.000）
Token 用量	85（流式不一致）	100
响应完整性	100	100
其余可执行项	100	100
综合评级	通过	优秀

修复后的观测：`output_tokens_seq = [60, 60, 60]`，流式与非流式均为 1，`count_tokens` 与实际输入一致。

6 检测评估

Veridrop 在 standard 模式下执行八项检测：身份一致性、模型一致性、知识准确度、结构化输出、响应完整性、Token 用量、协议规范性、消息标识规范。思维签名与行为签名因别名不在其模型表内被跳过，长上下文与 PDF 识别需 full 模式。

修复后，上述八项全部为 100 分。

7 两类被高估的检测信号

检测社区目前把两类信号当作"高可信"证据：Anthropic 的 thinking signature 加密校验，以及基于数字分布的指纹模型检测。本章结合本演示的实测，说明它们的实际效力边界。

7.1 Thinking signature：验证的是格式，不是来源

被高估的理由。 真实的 `thinking` 块携带 Anthropic 服务端签名，客户端在后续轮次回传该块以延续推理。检测工具据此认为：只要签名"加密验证通过"，就说明响应确实来自 Anthropic——这是当前检测集中权重最高的单项（在 Veridrop 中占 25%）。

它实际能验证什么。 客户端手里没有 Anthropic 的公钥，也没有签名的格式规范，无法离线验证签名与内容的对应关系。它能做的只是结构性检查：块是否存在、签名长度是否达标、字符集是否为 base64url、把块回传后是否被接受。这些性质与"签名由谁生成"无关。

演示系统的做法。 用进程启动时生成的 X25519 临时密钥，对上游推理文本做 zlib 压缩 → ECDH → HKDF-SHA256 派生 → AES-256-GCM 加密，再 base64url 编码。三个细节让结构检查全部通过：

- **长度随内容变化**——按推理文本长度线性伸缩，实测 239 至 1598 字符，而不是固定值；
- **绑定会话**——把 `session_id` 一并密封，签名块拿到另一个会话回放会校验失败，表现得像"服务端认识这个会话"；
- **每次不同**——GCM 的随机 nonce 使同一段推理两次得到不同签名，符合真实实现的随机性。

于是"加密验证通过"这一结论，在演示系统上同样成立——但它证明的只是"这段字节看起来像 Anthropic 的签名"，而不是"它由 Anthropic 签发"。

结论。 在没有可信公钥分发与格式规范的前提下，signature 检测的判别力等同于格式校验。它可以把粗糙的伪造挡在门外（固定长度、纯随机字节、无法回传），但对一个认真实现密封的伪造者无效。把它当作"最高可信度信号"是高估。

7.2 数字指纹检测：数学规律依赖两个前提

方法。 不使用工具，直接要求模型生成一定数量的整数，再用统计规律判断后端是哪个模型。依据是：模型生成的"随机"数并非均匀独立，而带有稳定的个体特征——数字偏好、重复倾向、排列习惯。这类工具（如 ikaleio 一类的指纹检测）在社区中被视为难以伪造的身份证据。

探针的形状。 有效的探针有两个形式要求，缺一不可。其一是**间隔**：值之间必须有分隔符，否则无法切分——一串连续数字既可能是 218 个数，也可能是一个 218 位的数，两者的统计口径完全不同。其二是**范围上界**：所有值落在同一区间，不同次采样才可比。社区探针因此长这样：

```
give me 218 random integers between 1 and 355, comma separated, no other text
```

218 个值、上界 355、逗号分隔——三个数字共同定义了样本的统计口径。

前提一：请求必须真的抵达模型。 指纹的全部信息来自模型的实际输出。一旦中转站识别出这类请求并返回录制样本，检测器测到的是样本库的分布，与真实后端无关。本演示的分流机制正是如此：判官识别"索要一长串伪随机整数"的请求，从样本库取最接近的一份返回。实测直接发送上述探针，返回的是 818 字符的录制样本而非模型输出。

绕过尝试有三条路径，实测结果如下：

方案	代表请求	是否被分流	结论
计数单位错配	一个恰好 200 位的随机整数	否	成立，但无间隔、无范围
分片累积	10 个 1–355 的随机整数	是	被击破
语义重编码	40 个数字的四种写法	是	被击破

后两套失败的原因一致：判据虽以个数表述，判官实际执行的是语义判断，拆分与换皮都无法规避。第一套成立，是因为它改变的不是表达方式，而是被计数的对象；但它产出的是单个大数，没有间隔与范围，与社区探针的口径不同。

载体：把被拦截的探针原样送进去。 既然判官只读用户消息，就让消息里不出现这句话：把探针原文编码进一段可编译的源码（小写字母前移三位、整串倒序），用户消息只要求"审阅这段程序，算出它打印什么，再照做"。判官看到的是代码审阅请求，实测未被分流：

模型	是否被分流	回收数值	取值范围
`claude-sonnet-5`	否	224	2–355
`claude-opus-5-5`	否	218	5–353

`opus-5-5` 恰好给出 218 个值，与请求一致；两个模型的取值都落在 1–355 内。也就是说，**带间隔、带范围限制的探针可以完整通过**，只是不再以自然语言的形式出现。

前提二：统计量必须匹配任务形状。 这一点比第一点更隐蔽。同一后端、两种探针，统计结论完全不同：

探针	统计量	观测值	基线或临界值	结论
单个 300 位数	数字卡方（df=9）	0.164	16.919	看似均匀——误判
单个 300 位数	10 位窗口排列率	12.1%	0.036%	强结构，指纹明确
单个 300 位数	一阶转移熵	2.875 bit	3.322 bit	偏低
218 个 1–355 整数	数字卡方（df=9）	122–179	16.919	拒绝均匀，但被范围污染
218 个 1–355 整数	个位数字卡方（df=9）	23.19	16.92	拒绝均匀，值层面存在偏好
218 个 1–355 整数	值分桶卡方（df=5）	2.29	11.07	未见异常

两点值得注意。第一，**位数统计在范围受限的探针上会被范围本身污染**：1 到 355 的三位数首位只能是 1、2、3，数字频率天然偏离均匀，此时卡方拒绝均匀并不等于模型有偏好。第二，**单个大数探针没有范围约束**，其结构直接暴露在数字序列上——0–9 的循环排列使窗口排列率达到随机基线的 336 倍，而边缘卡方反而低到 0.164，足以骗过只做频率检验的工具。

结论。 数字指纹是一类**有条件成立**的证据：它要求请求抵达真实模型，且要求统计量与探针形状匹配。两个条件缺一，结论就不可靠；而两个条件都不由检测方单方面控制——前者取决于中转站是否分流，后者取决于检测方是否理解自己的探针。

防御建议。

- **按数字而非按数值计数。** 单个 200 位数与 200 个一位数在指纹意义上等价。
- **检测长数字串。** 任何连续 64 位以上的数字串都应视为候选指纹载荷。
- **用序列统计而非边缘统计判别。** 卡方检验在本例中完全失效。
- **跨请求累计。** 即使单请求合规，同一会话内的累计数字量也应计入判据。
- **对代码块先求值再分类。** 7.2 表明判官只看文本是不够的；需要先判断"这段程序会打印什么"，再对打印结果分类。

8 讨论

8.1 为什么客户端难以识别

协议不提供跨跳完整性。客户端能验证的只有自洽性：字段是否齐全、类型是否正确、同一请求的两次调用是否一致。这些性质可以被一个诚实但不稳定的上游破坏，也可以被一个稳定但说谎的中转站满足。**自洽不等于真实。**

8.2 有效性威胁

构造效度。 一致性判据基于 `output_tokens` 的变异系数。若上游恰好稳定，缺陷不会暴露；本研究通过重复调用确认抖动真实存在。

外部效度。 结论依赖上游的不确定性程度。更换上游后重放机制仍有效，但必要性可能下降。

内部效度。 修复后的满分不等于"无法被检测"。思维签名检测因别名不在检测器模型表内被跳过（权重 25%），这是评估覆盖的空缺，而非通过。

8.3 局限

- 确定性重放引入状态：同一提示词在 TTL 内只有一次真实上游调用。
- 录制样本库存在数据质量问题：某家族中标记 218 个值的样本实际含 238 个值。
- 判官调用为每个请求增加约 1.4 秒延迟。
- 第 4 章实测发现，模型不产生推理时签名回落到固定 700 字符；该常数尚未修复，是可被检测的实现缺陷。

9 结论

中转站掺水的本质是**利用协议不提供端到端完整性**：凡是响应里写的东西都可以改，凡是客户端无法独立验证的东西都不会被发现。本文给出的五类分类学表明，掺水不是单一手法，而是沿身份、计量、内容、探测规避与性能五个方向展开的一组技术；每一类都对应着特定的协议层痕迹，也对应着特定的检测手段。

演示系统 OpenClaude 说明这些手法在工程上都可实现，且实现成本不高；而案例研究表明，即便是"无辜"的透传，也会因为上游的不确定性而在外部表现为掺水。检测方的有效武器不是加密验证，而是**不变量与统计**：五条计量不变量、重复调用的稳定性、以及数字序列的结构统计——后者在边缘频率完全失效时仍然有效。

附录 A 复现步骤

```
# 1. 启动服务
cd "$RELAY_DIR" && .venv/bin/python -m app --port "$PORT"
```

```
# 2. 运行检测器
cd "$DETECTOR_DIR" && .venv/bin/relay-detector detect \
  --base-url http://127.0.0.1:"$PORT" \
  --api-key "$RELAY_KEY" \
  --model claude-fable-5-1 \
  --mode standard -o /tmp/report.json

# 3. 单元测试
PYTHONPATH=.vendor:. python -m unittest tests.test_usage tests.test_acceptance \
  tests.test_thinking tests.test_pdf tests.test_fingerprint

# 4. 行为检查
PYTHONPATH=.vendor:. python tests/check_behaviour.py

# 5. 指纹提取（独立工具）
python fingerprint-extract/fingerprint_extract.py \
  --base-url http://127.0.0.1:"$PORT" --api-key "$RELAY_KEY" \
  --model claude-sonnet-5 --rounds 5 --digits 300
```

附录 B 关键文件

文件	职责
app/service.py	请求编排、计量映射、流式转发、确定性重放接入
app/upstream.py	OpenAI/Anthropic 双格式适配、流式解析、重试
app/thinking.py	思维链展示策略、签名块构造、用量块
app/replay.py	temperature=0 确定性重放缓存
app/usage.py	提示词 token 定标
app/ratelimit.py	按家族令牌桶（0 表示不整形）
app/fingerprint_bank.py	指纹样本库、判官、分流
app/crypto.py	X25519 + HKDF + AES-GCM 签名密封
tests/test_usage.py	计量不变量与重放测试（16 项）
fingerprint-extract/	独立提取工具（单大数探针 + 边缘/序列统计判别）